1. To design and implement a syntax checker and evaluator for a basic configuration language or subset of a scripting language that supports variable assignments and dynamic variable evaluation using the Bision parser generator and Flex lexical analyser.

CODE :

config.l

```
%{
#include "config.tab.h"
#include <string.h>
#include <stdlib.h>
%}

%%

[a-zA-Z_][a-zA-Z0-9_]* {
    yylval.str = strdup(yytext);
    return IDENTIFIER;
}

[0-9]+ {
    yylval.num = atoi(yytext);
    return NUMBER;
}

[-+*/=;\n] {
    return yytext[0];
}

[ \t]+ {

}

. {

}

%%

int yywrap() {
    return 1;
}
```

config.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex();
void yyerror(const char *s);

char vars[100][50];
int vals[100];
int var_count = 0;

int get_var(char *name) {
```

```
     for(int i = 0; i < var_count; i++) {
          if(strcmp(vars[i], name) == 0) {
               return vals[i];
          }
     }
     return 0;
}

void set_var(char *name, int val) {
     for(int i = 0; i < var_count; i++) {
          if(strcmp(vars[i], name) == 0) {
               vals[i] = val;
               return;
          }
     }
     strcpy(vars[var_count], name);
     vals[var_count] = val;
     var_count++;
}
%}

%union {
     int num;
     char* str;
}

%token <str> IDENTIFIER
%token <num> NUMBER
%type <num> expr

%left '+' '-'
%left '*' '/'

%%
program:
     program statement '\n'
     | program '\n'
     | statement '\n'
     | '\n'
     ;
statement:
     IDENTIFIER '=' expr ';' {
          set_var($1, $3);
          printf("Success: Assigned %s = %d\n", $1, $3);
          free($1);
     }
     | expr ';' {
          printf("Evaluated Result: %d\n", $1);
     }
     ;
expr:
     NUMBER               { $$ = $1; }
     | IDENTIFIER         { $$ = get_var($1); free($1); }
     | expr '+' expr      { $$ = $1 + $3; }
     | expr '-' expr      { $$ = $1 - $3; }
     | expr '*' expr      { $$ = $1 * $3; }
     | expr '/' expr      { $$ = $1 / $3; }
     | '(' expr ')'       { $$ = $2; }
     ;
%%
```

```
void yyerror(const char *s) {
     fprintf(stderr, "Syntax Error: %s\n", s);
}

int main() {
     yyparse();
     return 0;
}
```

Output :

```
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ bison -d
config.y
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ flex
config.l
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ gcc
config.tab.c lex.yy.c -o config

student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ ./config
x = 10 + 5;
Success: Assigned x = 15
x = 10;
Success: Assigned x = 10
y = 20;
Success: Assigned y = 20
x + y;
Evaluated Result: 30
```

2. Design and implement a syntax analyser and semantic validator that processes and evaluates nested parentheses balance and mathematical structural formatting using the Bison parser generator and Flex lexical analyzer.

CODE :

math.l

```
%{
#include "math.tab.h"
%}

%%

[0-9]+(\.[0-9]+)? {
    return NUMBER;
}

[a-zA-Z_][a-zA-Z0-9_]* {
    return ID;
}

[-+*/()\[\]{}\n] {
    return yytext[0];
}
```

```
[ \t]+ {

}

. {
    return INVALID;
}

%%

int yywrap() {
    return 1;
}
```

math.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(const char *s);

int valid = 1;
%}

%token NUMBER ID INVALID

%left '+' '-'
%left '*' '/'

%%

input:
      input line
    | line
    ;

line:
      expr '\n' {
          if (valid) {
              printf("Valid: Correct mathematical structure and
balanced symbols.\n");
          }
          valid = 1;
      }
    | error '\n' {
          valid = 1;
          yyerrok;
      }
    | '\n'
    ;

expr:
      NUMBER
    | ID
    | expr '+' expr
    | expr '-' expr
```

```
    | expr '*' expr
    | expr '/' expr
    | '(' expr ')'
    | '[' expr ']'
    | '{' expr '}'
    ;

%%

void yyerror(const char *s) {
    printf("Syntax Error: Invalid expression or unbalanced
parentheses/brackets.\n");
    valid = 0;
}

int main() {
    yyparse();
    return 0;
}
```

OUTPUT:

```
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ bison -d
math.y
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ flex math.l
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ gcc
math.tab.c lex.yy.c -o math
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ ./math
student@nitcse-OptiPlex-7000:~/24CS8017/LAB6$ ./math
(a+b) * {c - [d / e]}
Valid: Correct mathematical structure and balanced
symbols.
(a + b}
Syntax Error: Invalid expression or unbalanced
parentheses/brackets.
{X + [Y * (Z - 1)]}
Valid: Correct mathematical structure and balanced
symbols.
```